

0.95. Un micro Zelda — des salles, des murs, deux portes

Un autre jeu : le A se promène dans un monde de quatre salles. Il passe d'un écran à l'autre par les ouvertures du cadre ; les murs X l'arrêtent ; deux portes P l'envoient d'une salle à l'autre.

1. Ce qu'il faut comprendre

Un nouveau jeu, façon Zelda. Le héros, c'est le **A**. Le monde est plus grand que l'écran : **quatre salles**, deux de large et deux de haut. L'écran n'en montre qu'une à la fois. Quand le A sort par un bord, la salle d'à côté **remplace** celle-ci, et le A y entre par le bord opposé.

Ce qui vient d'avant : le pas toutes les 150 ms avec `chaque(150)`, la case d'arrivée `nx`, `ny` calculée avant de bouger (le 0.82), `lire()` pour savoir ce qu'il y a sur une case, « seulement sur une case vide » (`lire(nx, ny) == 0`, le 0.93.2), et `viderEcran()` (le 0.93.8).

Ce qui est nouveau ici (1) : où est-on dans le monde ? Deux variables : `sx`, la colonne de la salle (0 à gauche, 1 à droite), et `sy`, sa ligne (0 en haut, 1 en bas). Le numéro de la salle se calcule : `salle = sx + sy + sy`. Salle en haut à gauche : $0 + 0 + 0 = 0$. En haut à droite : $1 + 0 + 0 = 1$. En bas à gauche : $0 + 1 + 1 = 2$. En bas à droite : $1 + 1 + 1 = 3$. `ax` et `ay`, eux, disent où est le A **dans** la salle : colonne 0 à 19, ligne 0 à 16.

Ce qui est nouveau ici (2) : deux fonctions pour les murs. `mur(c, 1, n)` pose **n** X à la suite vers la droite : `mur(4, 4, 12)` pose des X de (4, 4) à (15, 4). `murDebout(c, 1, n)` fait pareil vers le bas : `murDebout(6, 3, 11)` pose des X de (6, 3) à (6, 13). Dans la boucle `for`, `i` vaut 0, 1, 2... jusqu'à `n - 1`, et chaque tour pose un X en `c + i` (ou en `1 + i`).

Ce qui est nouveau ici (3) : dessiner la salle. `dessinerSalle()` vide l'écran, puis : 1. le cadre en entier (lignes 0 et 16, colonnes 0 et 19) ; 2. **les ouvertures** : on efface le cadre là où une salle voisine existe. `sx == 0` : il y a une salle à droite, on ouvre les lignes 7, 8, 9 de la colonne 19. `sy == 0` : une salle en bas, on ouvre les colonnes 9 et 10 de la ligne 16. Et pareil à gauche et en haut. Une salle au bord du monde reste fermée de ce côté : le A ne peut pas sortir du monde. 3. les murs **propres à chaque salle**, avec des `if (salle == ...)`, et les deux portes ; 4. le numéro de la salle, sous le cadre.

Ce qui est nouveau ici (4) : sortir de l'écran. L'écran a les colonnes 0 à 19, et le terrain les lignes 0 à 16. Si l'arrivée est la colonne **20**, le A sort à droite : `sx = sx + 1`, et il entre dans la nouvelle salle en colonne **0**. Si l'arrivée est la ligne **17**, il sort en bas : `sy = sy + 1`, et il entre en ligne **0**.

Et à gauche ? Pourquoi 255 ? Un `uint8_t` va de 0 à 255, jamais en dessous. Quand `ax` vaut 0, `ax - 1` ne donne pas -1 : le nombre **fait le tour** et donne **255**, comme un compteur qui repart de la fin. Donc `nx == 255`, c'est « sorti à gauche » : `sx = sx - 1`, et il entre en colonne **19**. De même, `ny == 255`, c'est « sorti en haut » : il entre en ligne **16**. Ensuite, dans tous les cas : `dessinerSalle()`, et le A posé à sa nouvelle place.

Ce qui est nouveau ici (5) : les deux portes. Une porte **P** dans la salle 0, en (3, 12), et une dans la salle 3, en (15, 4). Marcher sur un P, c'est `lire(nx, ny) == ALPHABET[15]`. Si l'on est dans la salle 0, on part dans la salle 3 (`sx = 1`, `sy = 1`) ; sinon, on revient dans la salle 0. C'est un **raccourci** : à pied, il faut passer par la salle 1 ou la salle 2.

Pourquoi arriver À CÔTÉ de la porte, et pas dessus ? Sur le P, le A le cacherait : on ne verrait plus la porte. En (14, 4), juste à gauche du P en (15, 4), on la voit, et c'est au joueur de décider d'y retourner (DROITE).

L'ordre des tests compte. Dans la boucle : d'abord « hors de l'écran ? », car `lire(20, 8)` n'a pas de sens : la case n'existe pas. Puis « une porte ? ». Puis « une case vide ? ». Un X n'est dans aucun de ces cas : **il ne se passe rien**, le A reste où il est. C'est tout ce qu'il faut pour les murs.

Déroulons. Le A part de (9, 8), salle 0. DROITE : (10, 8), (11, 8)... jusqu'à (19, 8), l'ouverture du cadre. Encore DROITE : l'arrivée est (20, 8), hors de l'écran. `sx` passe à 1, `nx` à 0 : on est dans la **salle 1**, en (0, 8), et l'écran la montre. GAUCHE : l'arrivée est $0 - 1 = 255$: `sx` redevient 0, `nx` devient 19 : retour dans la salle 0, en (19, 8). Puis GAUCHE jusqu'en (4, 12), et encore GAUCHE : (3, 12) est la porte. On arrive dans la **salle 3**, en (14, 4). DROITE : (15, 4) est l'autre porte : retour dans la salle 0, en (4, 12).

Essaie : ajoute un mur dans la salle 2 avec `mur()` ou `murDebout()`, ou déplace une porte (change aussi la place d'arrivée, à côté d'elle).

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1521 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          72 o ← la console, à l'allumage — la mise en route de la console
chaque_minuteur() 124 o ← main() — une fonction fournie par le langage
mur()              38 o ← dessinerSalle() — ta fonction
murDebout()        42 o ← dessinerSalle() — ta fonction
viderEcran()       33 o ← dessinerSalle() — ta fonction
dessinerSalle()    312 o ← main() — ta fonction
main()            374 o ← démarrage — ta fonction
VBlank            8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage      28 o ← image() — attend l'image suivante
AttendreVBlank     16 o ← démarrage — attend que l'écran soit libre
AttendreAcces      12 o ← EffacerCases, EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase        10 o ← poser(), effacer(), lire() — trouve l'adresse d'une case de l'écran
EffacerCases       12 o ← effacer() — efface des cases
EcrireTexte        10 o ← texte() — écrit un texte, lettre par lettre
EcrireNombre       33 o ← nombre() — écrit un nombre en chiffres
CopierTuiles       85 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte       14 o ← démarrage — vide les deux cartes de l'écran
LireManette        37 o ← bouton() — lit les huit boutons
Tuiles            256 o ← CopierTuiles — 16 lettres sur 43
"SALLE"           5 o ← texte() — un texte gravé
Lettres gravées : AELPSX0123456789
```

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 8	102 o sur place, dans mur(), murDebout(), dessinerSalle(), main() + AdresseCase (10 o)
effacer()	× 10	93 o sur place, dans viderEcran(), dessinerSalle(), main() + AdresseCase (10 o),
EffacerCases (12 o)		
lire()	× 2	22 o sur place, dans main() + AdresseCase (10 o)
bouton()	× 4	20 o sur place, dans main() + LireManette (37 o)
texte()	× 1	11 o sur place, dans dessinerSalle() + EcrireTexte (10 o)
nombre()	× 1	10 o sur place, dans dessinerSalle() + EcrireNombre (33 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)
images()	× 1	2 o sur place, dans chaque_minuteur()

Tes fonctions appelées :

mur()	× 6	87 o sur place, dans dessinerSalle()
murDebout()	× 4	57 o sur place, dans dessinerSalle()
chaque_minuteur()	× 1	10 o sur place, dans main()
dessinerSalle()	× 3	9 o sur place, dans main()
viderEcran()	× 1	3 o sur place, dans dessinerSalle()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UN MICRO ZELDA ----
// Tout ce qui suit « // » sur une ligne est un COMMENTAIRE : la console
// ne le lit pas. Il n'est là que pour toi, pour expliquer le code.
//
// Les lettres : A = le héros, X = un mur, P = une porte.
// ALPHABET[0] : le A   ALPHABET[23] : le X   ALPHABET[15] : le P
// ALPHABET[n] : la n-ième lettre, en comptant à partir de 0 (pas de 1) :
//   A = 0, B = 1, C = 2, D = 3... O = 14, P = 15... W = 22, X = 23.
//
// Le monde : QUATRE salles, deux de large, deux de haut.
// L'écran n'en montre qu'une à la fois, comme dans Zelda.
//
//      sx = 0      sx = 1
// +-----+-----+
// | salle 0 | salle 1 |   sy = 0
// +-----+-----+
// | salle 2 | salle 3 |   sy = 1
// +-----+-----+
//
// sx : la colonne de la salle dans le monde (0 = à gauche, 1 = à droite).
// sy : la ligne de la salle dans le monde   (0 = en haut, 1 = en bas).
// Le numéro de la salle se calcule :
//   salle = sx + sy + sy
//   en haut à gauche : 0 + 0 + 0 = 0
//   en haut à droite : 1 + 0 + 0 = 1
//   en bas à gauche  : 0 + 1 + 1 = 2
//   en bas à droite  : 1 + 1 + 1 = 3
//
// Dans UNE salle, les cases vont :
//   colonne 0 (à gauche) à 19 (à droite) : 20 colonnes, toute la largeur de l'écran ;
//   ligne   0 (en haut) à 16 (en bas)   : 17 lignes pour le terrain.
//   La ligne 17, tout en bas de l'écran, sert à écrire le numéro de la salle.
//   Une case s'écrit (colonne, ligne) : (9, 8), c'est la colonne 9, la ligne 8.

// ---- Les variables ----
// Une variable, c'est une boîte qui garde un nombre, et qu'on peut changer.
// uint8_t : le genre de la boîte. Elle garde un nombre de 0 à 255,
//   jamais en dessous de 0, jamais au-dessus de 255.
// « = 9 » : le nombre qu'elle a au départ.
// Elles sont écrites ICI, hors de toute fonction : tout le programme les voit.

#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <effacer>    // efface des cases, ou tout le fond
#include <texte>      // écrit un texte à l'écran
#include <nombre>     // écrit un nombre en chiffres
#include <chaque>     // répond 1 toutes les n millisecondes
#include <bouton>     // lit un bouton de la manette
#include <lire>       // lit la tuile posée sur une case
```

2. UNE VARIABLE — AX

```
uint8_t ax = 9;           // le A, le héros : sa colonne...
```

3. UNE VARIABLE — AY

```
uint8_t ay = 8;           // ...et sa ligne, DANS la salle où il est. Il part de (9, 8).
```

4. UNE VARIABLE — NX

```
uint8_t nx = 0;           // la case d'ARRIVÉE, calculée AVANT de bouger (le 0.82) :
```

5. UNE VARIABLE — NY

```
uint8_t ny = 0;           // on regarde ce qu'il y a dessus, PUIS on décide de bouger ou non.
```

6. UNE VARIABLE — SX

```
uint8_t sx = 0;           // la salle : sa colonne dans le monde (0 ou 1)...
```

7. UNE VARIABLE — SY

```
uint8_t sy = 0;           // ...et sa ligne (0 ou 1). On part de la salle 0, en haut à gauche.
```

8. UNE VARIABLE — SALLE

```
uint8_t salle = 0;        // son numéro, 0 à 3, calculé dans dessinerSalle()

// ---- n murs X à la suite, vers la DROITE, à partir de (c, l) ----
// void : la fonction fait quelque chose, mais ne rend pas de nombre.
// (uint8_t c, uint8_t l, uint8_t n) : ce qu'on lui donne en l'appelant.
//   c : la colonne du premier X, l : sa ligne, n : combien de X.
// Exemple : mur(4, 4, 3) pose un X en (4, 4), (5, 4) et (6, 4).
```

9. UNE FONCTION — MUR()

```
void mur(uint8_t c, uint8_t l, uint8_t n) {
    // for (départ ; tant que... ; après chaque tour) :
    //   uint8_t i = 0 : i commence à 0 ;
    //   i < n          : on continue tant que i est plus petit que n ;
    //   i++            : après chaque tour, i gagne 1.
    // Avec n = 3 : i vaut 0, puis 1, puis 2. À 3, « 3 < 3 » est faux : on s'arrête.
    // La boucle fait donc n tours : n X.
    for (uint8_t i = 0; i < n; i++) {
        poser(c + i, l, ALPHABET[23]);    // un X, i cases à droite du départ ; la ligne ne change pas
        // mur(4, 4, 3) : i = 0 → (4, 4) ; i = 1 → (5, 4) ; i = 2 → (6, 4)
    }
}

// ---- n murs X à la suite, vers le BAS, à partir de (c, l) ----
// Exemple : murDebout(6, 3, 3) pose un X en (6, 3), (6, 4) et (6, 5).
```

10. UNE FONCTION — MURDEBOUT()

```
void murDebout(uint8_t c, uint8_t l, uint8_t n) {
    for (uint8_t i = 0; i < n; i++) {      // la même boucle que dans mur()...
        poser(c, l + i, ALPHABET[23]);    // ...mais c'est la LIGNE qui avance, pas la colonne
        // murDebout(6, 3, 3) : i = 0 → (6, 3) ; i = 1 → (6, 4) ; i = 2 → (6, 5)
    }
}

// ---- vider l'écran, comme dans le serpent (le 0.93.8) ----
// Les 18 lignes de l'écran (0 à 17) : chacune est effacée sur ses 20 cases.
```

11. UNE FONCTION — VIDERECRAN()

```
void viderEcran() {
    for (uint8_t l = 0; l < 18; l++) {    // l vaut 0, 1, 2... jusqu'à 17
        effacer(0, l, 20);                // effacer(colonne, ligne, combien de cases) :
                                           // 20 cases à partir de la colonne 0 : toute la ligne
    }
}

// ---- dessiner la salle (sx, sy) : tout l'écran ----
// On l'appelle au départ, et chaque fois que le A change de salle :
// l'ancienne salle disparaît, la nouvelle est dessinée à sa place.
// Elle ne dessine PAS le A : c'est fait après l'appel, là où on l'appelle.
```

12. UNE FONCTION — DESSINERSALLE()

```
void dessinerSalle() {
    viderEcran();                // on part d'un écran vide
    salle = sx + sy + sy;        // le numéro de la salle (voir le dessin, tout en haut)

    // 1. le cadre, en entier : les lignes 0 et 16, les colonnes 0 et 19
    mur(0, 0, 20);               // en haut : 20 X, de (0, 0) à (19, 0)
    mur(0, 16, 20);              // en bas : 20 X, de (0, 16) à (19, 16)
    murDebout(0, 0, 17);         // à gauche : 17 X, de (0, 0) à (0, 16)
    murDebout(19, 0, 17);        // à droite : 17 X, de (19, 0) à (19, 16)
    // Les coins, comme (0, 0), reçoivent deux X : l'un sur l'autre, on n'en voit qu'un.

    // 2. les OUVERTURES : on efface le cadre du côté où il y a une salle voisine.
    // Du côté où il n'y en a pas, le cadre reste : on ne sort pas du monde.
    // « == » compare deux nombres (est-ce égal ?) ;
    // « = » tout seul, lui, range un nombre dans une variable.
    if (sx == 0) {               // on est à gauche du monde : il y a une salle À DROITE
        effacer(19, 7, 1);       // 3 cases ouvertes dans la colonne 19 :
        effacer(19, 8, 1);       // les lignes 7, 8 et 9
        effacer(19, 9, 1);       // (une case à la fois : effacer() va vers la droite, pas vers le bas)
    }
    if (sx == 1) {               // on est à droite du monde : il y a une salle À GAUCHE
        effacer(0, 7, 1);        // les mêmes lignes, dans la colonne 0 :
        effacer(0, 8, 1);        // les deux ouvertures sont en face l'une de l'autre,
        effacer(0, 9, 1);        // le A sort par l'une et entre par l'autre
    }
    if (sy == 0) {               // on est en haut du monde : il y a une salle EN BAS
        effacer(9, 16, 2);       // 2 cases ouvertes dans la ligne 16 : les colonnes 9 et 10
    }
    if (sy == 1) {               // on est en bas du monde : il y a une salle EN HAUT
        effacer(9, 0, 2);        // les mêmes colonnes, dans la ligne 0
    }
    // Exemple : la salle 0 (sx = 0, sy = 0) est ouverte à droite et en bas ;
    // la salle 3 (sx = 1, sy = 1) est ouverte à gauche et en haut.

    // 3. les murs DANS la salle : chaque salle a les siens.
    // Un seul de ces quatre « if » est vrai : celui de la salle où l'on est.
    if (salle == 0) {
        mur(4, 4, 12);           // une barre en haut : 12 X, de (4, 4) à (15, 4)
        poser(3, 12, ALPHABET[15]); // la PORTE de la salle 0, un P en (3, 12)
    }
    if (salle == 1) {
        murDebout(6, 3, 11);     // une colonne de (6, 3) à (6, 13) : on passe en haut ou en bas...
        murDebout(13, 1, 11);    // ...une autre de (13, 1) à (13, 11) : on passe en bas seulement
    }
    if (salle == 2) {
        mur(3, 5, 6);            // une barre de (3, 5) à (8, 5)...
        mur(11, 11, 6);          // ...et une autre de (11, 11) à (16, 11)
    }
    if (salle == 3) {
        mur(3, 8, 12);           // une barre au milieu : de (3, 8) à (14, 8)
        poser(15, 4, ALPHABET[15]); // la PORTE de la salle 3, un P en (15, 4)
    }

    // 4. sous le cadre, sur la ligne 17 : le numéro de la salle
    texte(0, 17, "SALLE");       // le mot, à partir de la case (0, 17)
    nombre(6, 17, salle);        // le nombre, à partir de la case (6, 17)
}

// ---- main() : c'est ICI que la console commence ----
```

13. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // ---- Le départ ----
    dessinerSalle();                // la salle 0 (sx = 0, sy = 0)
    poser(ax, ay, ALPHABET[0]);    // le A à sa place, en (9, 8)

    // while (true) : « tant que vrai » : la boucle ne s'arrête jamais.
    // Tout ce qui est entre ses accolades { } recommence, encore et encore.
    while (true) {
        image();                    // on attend l'image suivante : 60 par seconde

        // ---- le A bouge avec la croix, un pas toutes les 150 ms (le 0.93.1) ----
        // chaque(150) est vrai une fois toutes les 150 millisecondes,
        // faux le reste du temps : le A fait environ 6 pas par seconde.
        if (chaque(150)) {
            // 1. la case d'ARRIVÉE (le 0.82). On part de la case où l'on est...
            nx = ax;
            ny = ay;
            // ...et une flèche la change d'une case.
            // bouton(DROITE) est vrai tant que la flèche droite est tenue.
            // else if : « sinon, si... ». Dès qu'une flèche est trouvée, on ne
            // regarde pas les autres : une seule à la fois, pas de pas en diagonale.
            if (bouton(DROITE)) {
                nx = ax + 1;          // une colonne à droite
            } else if (bouton(GAUCHE)) {
                nx = ax - 1;          // une colonne à gauche (et 0 - 1 donne 255 !)
            } else if (bouton(BAS)) {
                ny = ay + 1;          // une ligne plus bas
            } else if (bouton(HAUT)) {
                ny = ay - 1;          // une ligne plus haut (et 0 - 1 donne 255 !)
            }
            // Pourquoi 255 ? Un uint8_t ne descend jamais sous 0 : 0 - 1 « fait
            // le tour » et donne 255, comme un compteur qui repart de la fin.

            // 2. une flèche est tenue : l'arrivée n'est pas la case où l'on est.
            // « != » veut dire « différent de ». Sans flèche, nx == ax et
            // ny == ay : rien à faire, on saute tout ce bloc.
            if (nx != ax || ny != ay) {
                // « || » veut dire « ou » : il suffit d'UNE des quatre conditions.
                if (nx == 20 || nx == 255 || ny == 17 || ny == 255) {
                    // 3. ...et l'arrivée est HORS de la salle : on change de SALLE.
                    // Ce test vient EN PREMIER : lire(20, 8) n'aurait pas de sens,
                    // la case (20, 8) n'existe pas.
                    // On ne peut sortir que par une ouverture : ailleurs, le cadre
                    // X est sur le chemin, et le A s'arrête avant (le cas 6).
                    if (nx == 20) {    // sorti à droite (19 + 1 = 20) :
                        sx = sx + 1;    // la salle de droite (sx : 0 → 1)...
                        nx = 0;        // ...où l'on entre par la gauche, en colonne 0
                    }
                    if (nx == 255) {   // sorti à gauche (0 - 1 = 255) :
                        sx = sx - 1;    // la salle de gauche (sx : 1 → 0)...
                        nx = 19;       // ...où l'on entre par la droite, en colonne 19
                    }
                    if (ny == 17) {    // sorti en bas (16 + 1 = 17) :
                        sy = sy + 1;    // la salle d'en bas (sy : 0 → 1)...
                        ny = 0;        // ...où l'on entre par le haut, en ligne 0
                    }
                    if (ny == 255) {   // sorti en haut (0 - 1 = 255) :
                        sy = sy - 1;    // la salle d'en haut (sy : 1 → 0)...
                        ny = 16;       // ...où l'on entre par le bas, en ligne 16
                    }
                    // La ligne (ou la colonne) qui ne change pas reste la même :
                    // sorti à droite en ligne 8, on entre à gauche... en ligne 8.
                    ax = nx;           // le A prend sa place dans la nouvelle salle
                    ay = ny;
                }
            }
        }
    }
}
```

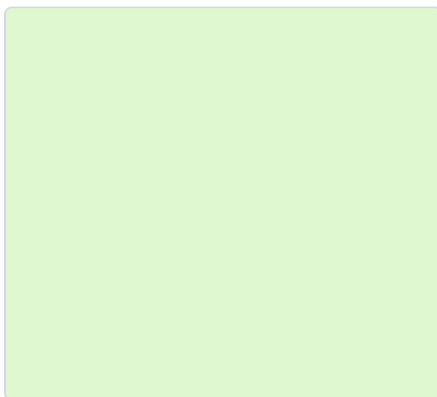
```

dessinerSalle();          // la nouvelle salle remplace l'ancienne
poser(ax, ay, ALPHABET[0]); // et le A y est dessiné
} else if (lire(nx, ny) == ALPHABET[15]) {
    // 4. ...l'arrivée est une PORTE P : elle mène à l'autre porte.
    // lire(colonne, ligne) rend la lettre posée sur cette case.
    // On n'arrive pas SUR la porte, mais à côté : on la voit encore,
    // et c'est au joueur de décider d'y retourner.
    if (salle == 0) {      // la porte de la salle 0 mène à la salle 3...
        sx = 1;           // (sx = 1, sy = 1 : en bas à droite)
        sy = 1;
        ax = 14;          // ...à côté de sa porte, en (15, 4) : une case à gauche
        ay = 4;
    } else {               // sinon, on est dans la salle 3 : sa porte ramène à la salle 0...
        sx = 0;           // (sx = 0, sy = 0 : en haut à gauche)
        sy = 0;
        ax = 4;           // ...à côté de sa porte, en (3, 12) : une case à droite
        ay = 12;
    }
    // C'est un RACCOURCI : à pied, de la salle 0 à la salle 3, il faut
    // passer par la salle 1 ou par la salle 2.
    dessinerSalle();       // l'autre salle remplace celle-ci
    poser(ax, ay, ALPHABET[0]); // et le A y est dessiné
} else if (lire(nx, ny) == 0) {
    // 5. ...l'arrivée est VIDE (la tuile 0, un espace) : le A y va.
    effacer(ax, ay, 1);    // le A quitte sa case (elle redevient vide)...
    ax = nx;               // ...prend la case d'arrivée...
    ay = ny;
    poser(ax, ay, ALPHABET[0]); // ...et y est dessiné
}
// 6. ...sinon, l'arrivée est un X : aucun des cas du dessus.
// Il ne se passe rien, le A reste où il est. C'est ça, un mur :
// il n'y a même pas besoin d'écrire un « if » pour lui.
}
}
}
}

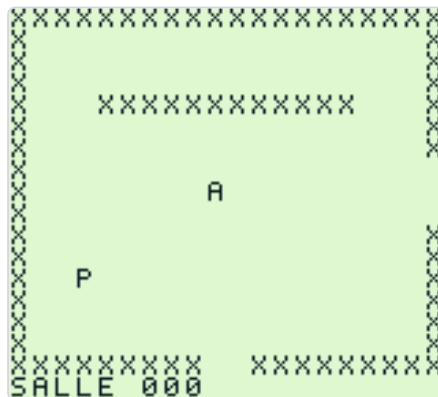
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



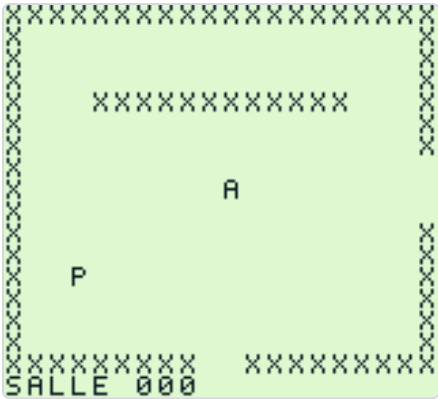
la toute première image — la console vient de démarrer — image 1



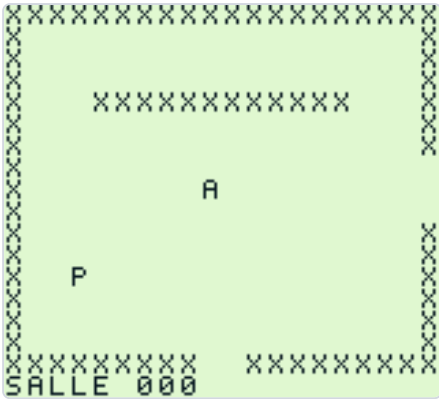
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

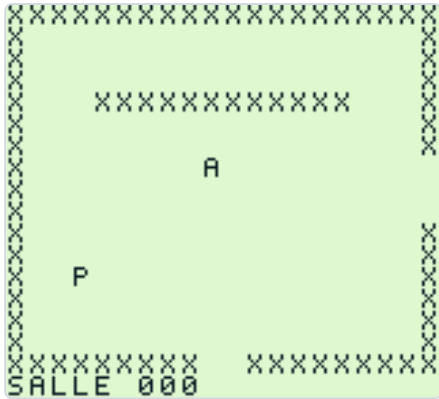
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



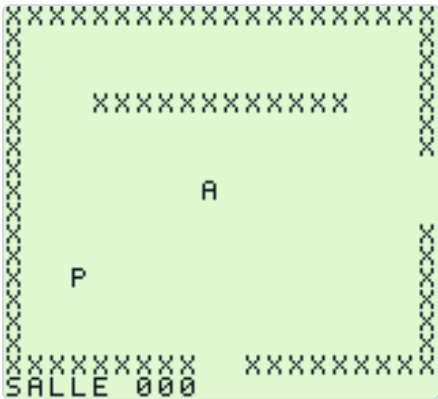
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Le A passe d'une salle à l'autre par les ouvertures du cadre ; les X l'arrêtent ; le P de la salle 0 mène à la salle 3, et celui de la salle 3 ramène à la salle 0.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la salle 0 : le cadre, le A en (9, 8), la porte P en (3, 12)
- ✓ HAUT : le mur X en (9, 4) arrête le A en (9, 5) (le A en ligne 5)
- ✓ sorti à droite, le A entre dans la salle 1 par la gauche
- ✓ sorti à gauche (255), il revient dans la salle 0, en colonne 19
- ✓ la porte de la salle 0 mène à la salle 3, à côté de sa porte
- ✓ la porte de la salle 3 ramène à la salle 0

Ensuite : **0.95.1. Un micro Zelda — l'écran glisse dans le sens du A** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++